

읽지 못하면, 읽지 못했다고 말하는 OCR

소비기한 추출 파이프라인과 중고거래 잔여기한 안내

4코어 CPU · GPU 없음 · 오프라인 환경에서의 경량 OCR 설계와 검증

제3회 ITDA 연합학술제 본선 | [CODE]_서브웨이

민섭 · 서현 · 병철 · 승아 · 서영

| 문제 정의

무엇을, 어떤 제약 아래에서 풀었는가



출발점 : 제약이 설계를 결정했다

3,352장

정답 라벨 없음

상품 뒷면 사진만 제공. 정확도를 잴 기준
부터 직접 만들어야 했다

4코어 CPU

GPU 없음 · 오프라인

채점 서버 Ubuntu 22.04, RAM 8GB.
무거운 모델·외부 API 사용 불가

2,500초

500장 제한 시간

초과하면 결과 파일이 나오지 않아 정확
도까지 0점. 장당 5초가 상한

첫 파이프라인(EasyOCR)은 4코어에서 **장당 5.8초, 500장 2,905초** 로 한도를 넘겼다

정확도를 올리기 전에 "**끝까지 돌아가는가**" 부터 풀어야 하는 문제였다

그래서 모든 설계 결정에 **정확도와 시간을 함께** 기록했다

| 데이터와 검증 설계

정답이 없는 데이터에서 정확도를 재는 법

2

검증 설계 : 규칙을 먼저 정하고, 시험지를 봉인했다

01 해석 규칙을 먼저 문서로 확정

라벨러는 포장에 찍힌 그대로만 입력. 해석(연·월·일 순서, 2자리 연도)은 도구가 규칙으로 처리 → 라벨과 모델이 같은 규칙을 쓴다

02 3,352장 전수 직접 라벨링

5명 분담, 자체 제작 라벨링 도구. 오답 530장을 사람이 다시 보고 라벨 오류 23장 수정

03 약점 유형만 골라 추가 촬영

도트 인쇄·각인·두 날짜 병기 위주 489장 촬영, 389장 라벨링. 한 번도 본 적 없는 시험지로 사용

데이터 분리

500

2,852

봉인 500장 학습·규칙 도출에 쓰지 않음. 측정 전용

판정용 2,852장 모든 채택·기각 판정

채택 기준

- 새로 맞힌 장(회복)과 새로 틀린 장(퇴보)을 따로 센다
- 퇴보가 회복의 1/3 을 넘으면 점수가 올라도 기각
- $\pm 1\%p$ 는 잡음으로 보고 근거로 쓰지 않는다

정답 라벨도 검증 대상 : 고친 라벨 23장 중 **16장은 모델이 맞은 쪽** 이었다

| 아키텍처 설계

왜 이 구조인가

3

전체 파이프라인 : 쉬운 사진은 한 번에, 어려운 사진에만 비용을



조건이 맞을 때만 동작하는 단계

장치	발동 조건	동작	효과
이웃 줄 재탐지	후보 줄 옆에 크기 비슷한 숫자 줄	그 주변만 원본 해상도로 다시 탐지	병기 +5 / -1
도트 인쇄 폴백	후보 없음	점을 이어 붙이는 전처리 후 재시도	+17 / -0
파인튜닝 인식기	그래도 후보 없음	우리 라벨로 학습한 인식기로 재시도	+7 / -0
시간 예산 가드	예상 소요 2,000초 초과	폴백 생략 → 2패스 생략 → NONE	완주 보장

딥러닝은 "읽기" 만, 소비기한을 고르는 판단은 설명 가능한 규칙 이 맡는다

설계 사고 : 오답을 "정답이 사라진 단계" 로 진단했다

예선 구성의 오답 603장 (3,352장 전체 기준) 마다, 정답 날짜가 파이프라인 어느 단계까지 살아 있었는지 추적

183

못 읽음

탐지 실패 · 전부 미인식

원본 재탐지
작은 사진 확대

290

잘못 읽음

인식기가 숫자를 오독

후처리 규칙 보강
파인튜닝 풀백

111

읽었다가 지움

다시 읽기가 정답을 덮어씀

"교정만, 삭제 금지"
유지 규칙

19

잘못 고름

가짜 후보가 낮은 날짜로 승리

이중 해석 억제

단계마다 처방이 다르다

- "읽었다가 지움" 111장은 모델이 아니라 코드 로직의 문제 → 규칙 하나로 판정용 2,852장 +45 / -4
- "못 읽음" 은 해상도를 올려도 회복되지 않았다 (5가지 구성 전부 $\pm 1\%p$, 시간만 +30~50%) → 원인은 탐지가 아니라 인식 품질

오답을 뭉뚱그려 "모델을 키우자" 가 아니라, **원인별로 가장 싼 처방** 을 골랐다

후처리 규칙 : 소비기한을 다른 숫자와 구분하는 법

구분 대상	실제 예	규칙	근거
품목보고번호 · 바코드	20130628332176	9자리 이상 숫자열은 통째로 제거	앞 8자리가 완전한 날짜 형식
영양성분표 · 중량	78 7 5 31 / 30.4 GRAMS	신뢰 등급 : 완전 날짜 > 공백·압축 > 연월 > 월일	잡음이 이긴 오답 15장에서 도출
제조일자 · 유통기한	25.06.26 / 26.06.25	가장 좋은 등급 안에서 가장 늦은 날짜	소비기한 ≥ 유통기한 ≥ 제조일
시각 · 로트번호	13:11 / A03 / L8	날짜 패턴에서 제외, 옆 글자를 '일' 자리로 읽지 않음	도트 인쇄 제품에 혼함
연·월·일 순서	30.07.26 / 24/12/21	년월일 우선. 무효·2028년 이후면 일월년	라벨 : 년월일 344 · 일월년 102
범위 밖 연도	2033, 2009	2017 ~ 2031 만 인정	전수 라벨 최소 2017

미인식(NONE) 기준

후보가 하나도 없으면 추측하지 않는다. 연도나 일만 없으면 그 칸만 NONE (NONE-10-14, 2027-07-NONE)

라벨과 모델이 같은 규칙

운영진 확정 : 모호한 표기는 팀이 문서화한 규칙으로 채점. 규칙 문서 하나를 라벨링 도구와 파서가 공유

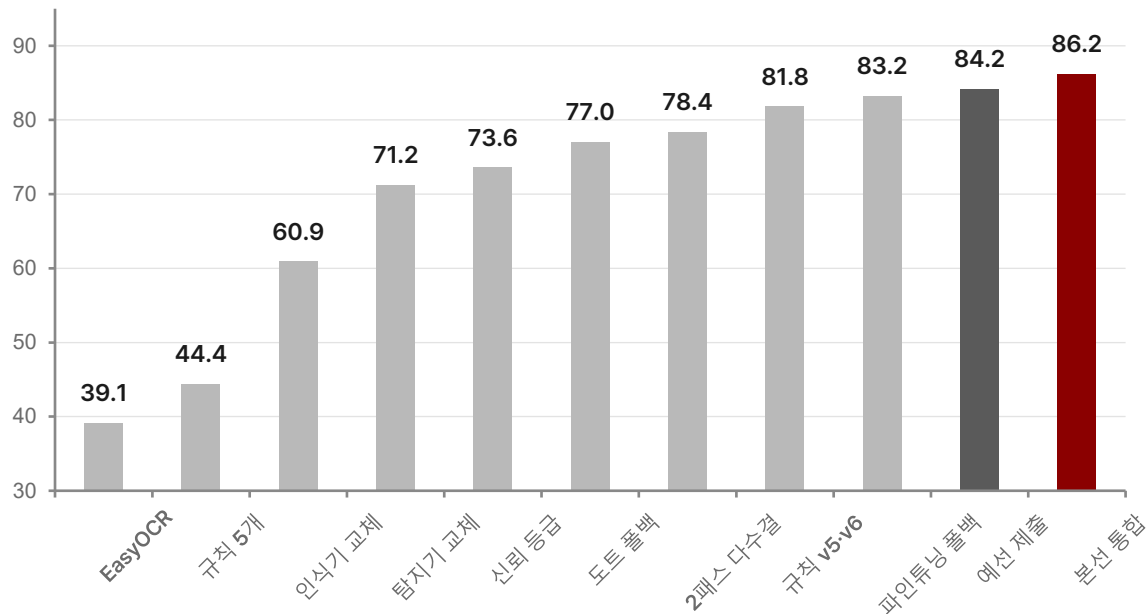
규칙마다 어떤 오답에서 나왔는지 와 회복·퇴보 수 가 기록돼 있다

| 성능과 속도 검증

무엇을 채택하고 무엇을 버렸는가

4

정확도 : 봉인 500장 86.2%, 필드 평균 90.8%



* 앞 세 단계는 개발용 133장, 이후는 500장 기준

봉인 500장 · 완전일치

86.2%

예선 제출본 84.2% → +2.0%p

구분	정확도
필드 평균 (채점 방식)	90.8%
작은 사진 (≤700px)	83.7%
중간	89.5%
큰 사진	86.7%
두 날짜 병기	78.4%
미인식률	3.0%

판정용 2,852장에서도 **82.3 → 86.7%** : 튜닝에 쓴 셋과 봉인 셋이 함께 올랐다

모델·대안 비교 : 점수가 올라도 기각한 것들

시도	결과 (회복 / 퇴보)	판단	이유
탐지·인식기 PP-OCrv5 mobile 로 교체	39.1 → 67.7%, 5.8 → 1.8초	채택	속도 한도와 정확도를 동시에 해결
2패스 "교정만, 삭제 금지"	+45 / -4	채택	코드 로직 결함 수정
본선 규칙 합계 (위 포함 7개)	+102 / -7	채택	병기 64 → 78%
이웃 줄 원본 재탐지	병기 +5 / -1, 그 외 0 / 0	채택	게이트로 비용을 병기에만
파인튜닝 인식기를 1순위로	+86 / -64 (+0.8%p)	기각	맞던 답을 혼돈다 → 폴백으로만 사용
합성 도트 데이터 3,000장 학습	76.8% (정밀도 84 → 81%)	기각	진짜 글씨를 대충 읽게 됨
탐지 해상도 상향 (5구성)	±1%p, 시간 +30~50%	기각	원인이 해상도가 아님
한국어 인식기 2차 의견	+14 / -1 → 최신 규칙 위 +7 / -2	기본 꿈	5장보다 시간 여유 (장당 +0.3~0.7초)
회전 · 탐지 임계 완화 재시도	회복 0, 엉뚱한 값 6	기각	미인식을 오답으로 바꿀 뿐

파인튜닝은 +0.8%p 였지만 **퇴보 64장** 때문에 버렸다. 채택 기준이 점수보다 먼저다

속도·효율 : 한도의 60%, 그리고 완주 보장

약 3초

장당 처리 시간

4코어 고정 측정

1,500초

500장 환산

한도 2,500초의 60%

1.25GB

최대 메모리

한도 8GB (40장 실측)

0원

학습 비용

무료 Colab T4 만 사용

속도를 만든 결정

- 탐지는 640px 축소본에서 장당 1회 (탐지 비용 = 픽셀 수)
- 인식은 큰 글자부터, 낱파를 찾으면 잔글씨 생략
- 8개씩 배치 인식 (16개 498 → 321ms)
- 다시 읽기는 후보 줄 크롭만, 풀백은 후보 없는 장에만

시간 예산 가드 : 3단계 자동 경량화

최근 20장 속도로 총 소요를 예측, 2,000초를 넘길 것 같으면

- 1 재시도 단계 생략
- 2 다시 읽기 생략
- 3 남은 장 NONE, 결과 파일은 반드시 생성

채점 서버 속도를 모르는 상황에서 "느리면 0점" 을 구조로 막았다

실행 화면 : 실제 파이프라인 출력



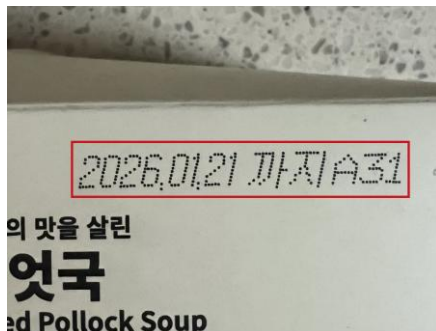
final_date 날짜 후보 (패턴 - 경로 - 신뢰도)
2026-07-02 2026-07-02 YYYY - p2 - 0.98
 처리 2.97초 (CPU 4스레드)
 빨간 인쇄 - 1패스 후보 → 2패스 다수결로 확정

일반 인쇄



final_date 날짜 후보 (패턴 - 경로 - 신뢰도)
2026-06-25 2025-06-26 YY - p2 - 0.78
 2026-06-25 YY - p2 - 0.78
 처리 4.73초 (CPU 4스레드)
 제조·소비 병기 - 두 후보 중 낮은 날짜(소비기한) 선택

제조·소비 병기



final_date 날짜 후보 (패턴 - 경로 - 신뢰도)
2026-01-21 2026-01-21 CMP - erode2/p1@1024 - 0.89
 처리 12.67초 (CPU 4스레드)
 도트 인쇄 - 기본 인식 실패 → 점 및 한자 후 재시도 후 화수

도트 인쇄 (폴백)



final_date 날짜 후보 (패턴 - 경로 - 신뢰도)
NONE 후보 없음 → 주석하지 않고 미인식으로 출력
 처리 21.32초 (CPU 4스레드)
 확산 없음 → 미인식(NONE) - 전당에서는 사용 확인 대기 필요

미인식 → 판매자 입력

- 빨간 상자 = 선택된 날짜 줄, 회색 상자 = 다른 후보. 아래 패널은 파이프라인이 실제로 낸 값·후보·처리 시간
- 병기 사진 : 25.06.26(제조)과 26.06.25(소비)를 모두 읽고 낮은 날짜를 선택
- 네 번째 : 후보가 없으면 날짜를 지어내지 않고 NONE. 게시 화면에서는 빈칸으로 두고 판매자가 입력한다

재현 : 새 환경에서 설치 → 가중치 다운로드 → Run All 검증, Ubuntu·Python 3.10·오프라인 자동 테스트 통과

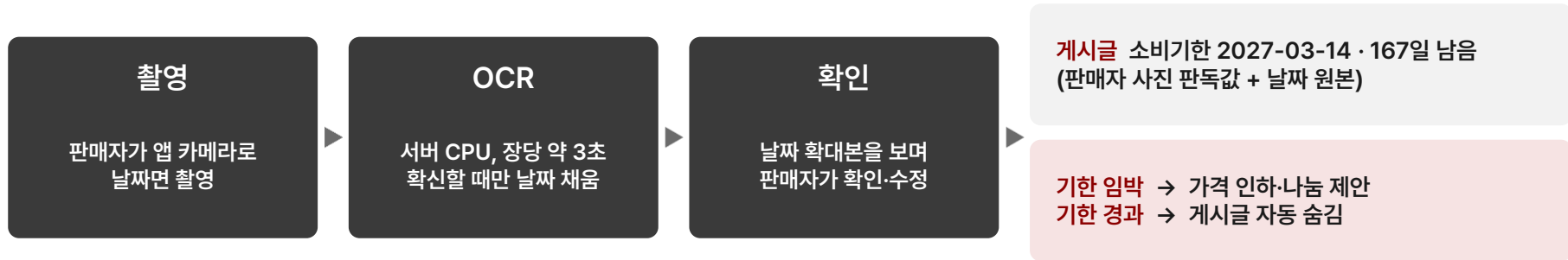
| 도메인 적용과 운영

중고거래 식품 게시글의 잔여기한

5

적용 도메인 : 중고거래 식품, 잔여기한을 거래 정보로

명절 선물세트가 중고거래로 : 당근 추석 연휴 선물세트 거래량 **+50%** (전년 대비). 그런데 소비기한은 판매자가 손으로 적는다



이 구조의 핵심

판매자가 마지막에 확인하므로, 모델이 틀려도 한 번 고치면 끝난다. 그래서 정확도는 **안전성이 아니라 자동 입력 비율**을 결정한다. 확신이 없으면 빈칸(NONE)으로 두니 지금보다 나빠지지 않는다

속이는 판매자는?

- 사기 방지 장치가 아니다. 스티커 위조·다른 상품 촬영은 기존 사후 모니터링 몫
- "확인됨" 대신 판독값과 날짜 원본을 보여 줘, 위조에 신뢰를 빌려주지 않는다
- 원본·촬영 시각·수정 이력은 신고 시 증거로 보존

당근은 이미 사진에서 인증마크를 OCR로 찾는다. 우리는 같은 사진에서 **소비기한을 찾아 날짜로 확정** 한다

운영 구조와 비용 : 이미 도는 OCR 옆에 CPU 한 대

항목	내용
1차 대상	건강기능식품 카테고리 (미개봉·소비기한 이내, 필수 기재)
규모	월 약 2.4만 건 (20개월 게시글 48만 건, 식약처 자료)
처리	게시할 때 1장, 장당 약 3초 → 하루 800장 ≈ CPU 40분
모델	21MB, GPU 없음 · 서버 또는 폰 온디바이스
판정	기한 경과로 읽히면 재확인 요청 (자동 차단 안 함)

비용과 가치

- 비용 ≈ 0 : 라이선스 0원, GPU 0대, 서버 CPU 일부
- 가치 : 잔여기한 안내·검색, 입력 편의, 기한 경과 글 자동 숨김
- 인력 절감으로 부풀리지 않는다 : 소비기한 위반은 전체 위반의 5% (725 / 14,729건)

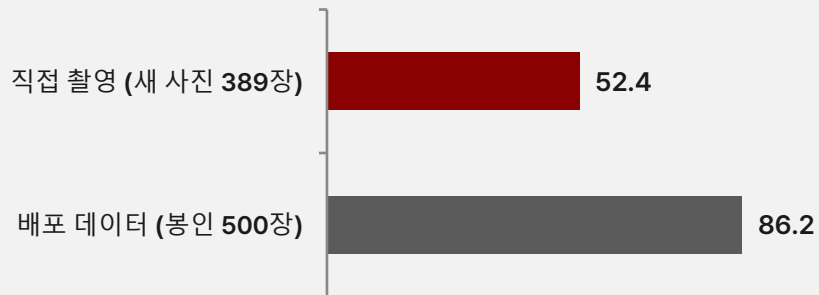
자동 입력 기준 (판매자 폰 사진을 흉내 낸 직접 촬영 389장 실측)

자동 입력 기준	채워지는 비율	채운 값의 정확도	틀린 값을 받는 판매자
읽히면 무조건 채움	70%	70%	21%
확신도 0.7 이상만	45%	77%	10%
확신도 0.9 이상만	17%	88%	2%

판매자가 고친 값이 **공짜 라벨** 이 되어, 쓸수록 자동 입력 비율이 오른다

한계 : 판매자 폰 사진에서는 52%

직접 촬영 389장 (팀원 4명, 약점 유형 위주)



남은 한계 세 가지

- 남은 날을 많게 읽는 경우 4.9% (중앙 +63일) → 제품 유형별 최대 기한을 넘으면 판독 불가
- 스티커 위조·다른 상품 촬영은 못 막는다 → 사후 모니터링 뭉
- 선물세트는 날짜가 여러 개 → 상품별로 찍고 가장 이른 날짜를 표시

보완 계획

01

확신할 때만 채운다

빈칸이면 지금처럼 손으로 입력. 손해가 없다

02

판매자 수정값으로 재학습

고칠 때마다 라벨이 생겨 자동 입력 비율 상승

03

차단 대신 안내

재확인만 요청하고 최종 판단은 기존 검토에

새 사진은 찍은 사람에 따라 44 ~ 70% : 촬영 안내(날짜를 가운데, 가까이)가 정확도를 좌우한다

확장성 : "제품에 찍힌 기한" 을 읽는 모든 곳

적용 분야	읽는 값	바꿀 것	그대로 쓰는 것
가공식품 · 명절 선물세트 카테고리	소비기한	대상 카테고리, 상품별 촬영	파이프라인 전체
품목보고번호 인식 → 제품 조회·회수 확인	품목보고번호	9자리 숫자 마스킹 규칙 재설계	탐지 · 인식 (정확도 미측정)
제조 현장 출하 전 날짜 인쇄 확인	소비기한	대조 대상 (공장의 기대값)	파이프라인 전체
의약품 · 화학 제품 출하 표시물	제조번호 · 사용기한	표기 형식 규칙	읽기 · 대조 · 예외 처리

왜 옮겨 쓸 수 있는가

- 딥러닝 모델은 "글자를 읽는 일" 만 하고, 도메인 지식은 전부 규칙 문서 한 장에 있다
- 읽기 → 기준값과 대조 → 예외만 사람에게 : 이 세 단계는 분야가 바뀌어도 같다
- 사람(판매자·검수자)이 확정한 값이 다시 학습 데이터가 되는 순환 구조

규칙 계층만 바꾸면 되는 구조 : 모델을 다시 만들 필요가 없다

정리

1

제약에서 출발

4코어 CPU · 2,500초 안에서 끝까지 돌아가는 구조. 봉인 500장 86.2%, 한도의 60%

2

검증을 먼저

전수 라벨링, 봉인 시험지, 회복·퇴보 기준. 점수가 올라도 퇴보가 크면 버렸다

3

틀리는 방식을 설계

확신이 없으면 NONE, 판매자가 직접 입력. 정확도는 안전성이 아니라 자동 입력 비율이 된다

부록 : 날짜 해석 규칙표 (모호한 표기)

상황	해석	근거 (전수 라벨 3,352장)
4자리 연도가 앞 / 뒤	앞이면 년/월/일, 뒤면 일/월/년. 구분자 종류는 무관	연도 앞 2,217 · 연도 뒤 362
2자리 세 개 (26.03.20)	년/월/일 우선. 무효이거나 연도 2028 이상이면 일/월/년	년월일 344 : 일월년 102
영문 유통 문구 (BBD, BEST BEFORE, EXP)	같은 사진에 있으면 일/월/년 우선	수입품 표기 관행
공백 구분 (30 12 23)	일/월/년	전부 수입품
영문 월 (04-Jul-21, 22OCT2021)	월 위치 확정, 4자리는 연도	148장
월/일/년 (06/18/23)	일/월/년이 불가능할 때만 (가운데 13 이상)	확정 미국식 2장
연월만 / 월일만	YYYY-MM-NONE / NONE-MM-DD	운영진 확정
날짜가 여럿	가장 좋은 등급 안에서 가장 늦은 날짜	병기 345장 (10.3%)

해석 기본값은 감이 아니라 **직접 단 라벨의 통계** 로 정했다

부록 : 측정 수치 한눈에

측정셋	구성	완전일치	필드평균	비고
봉인 500장	예선 제출본 (9/13)	84.2%	88.8%	미인식 17
봉인 500장	본선 통합본 (9/28)	86.2%	90.8%	미인식 15, 정밀도 88.9%
판정용 2,852장	예선 구성	81.9%		라벨 점검 후 82.3%, 병기 64.1%
판정용 2,852장	본선 최종 (한국어 2차 의견 끄)	86.7%		라벨 점검 후, 병기 78.3%
판정용 2,852장	통합본 (9/24, 규칙 4개 전)	85.1%	89.8%	예선 대비 +97 / -6
직접 촬영 389장	본선 통합본	52.4%	62.8%	약점 유형 위주, 미인식 22%

- 완전일치 : 연·월·일이 모두 맞은 사진 비율
- 필드평균 : 연·월·일을 따로 채점한 평균 (운영진 채점 방식)
- 회복 / 퇴보 : 이전 버전 대비 새로 맞힌 장 / 새로 틀린 장

모든 수치의 예측 파일과 실험 기록은 [저장소 results/ · docs/실험_기록.md](#)에 있다